Lecture plan · for the lecturer

# Hashing, passwords and authentication

Cyber security · University · Lecture 3

**Time**
60 min

**Materials**
Projector or board; The lecture slides and the seminar sheet from the kit; Phones or laptops with Darsi, for the lab

## Learning goals

- State hash properties.
- Store passwords properly.
- Choose a second factor.

## Activity: Crack a fake table

1. Warm-up (5 min): بأي سرعة؟
2. Teach (10 min): Show the avalanche effect and the three resistance properties.
3. Crack (25 min): Teams run a dictionary attack on fast, salted and Argon2id tables of synthetic users.
4. Decide (15 min): Write a one-paragraph storage and MFA policy for the university.
5. Wrap-up (5 min): Students write the muddiest point of the lecture and one question; read them before next week.

## Discussion

- How many accounts fell in each table?
- Why do salts matter even with slow hashes?
- Which factor would you require for staff?

## Exit ticket

- Write the storage line you would use for one password.

**Try it in your next lecture**
Give teams a small table of fake users hashed three ways; they time a dictionary attack on each and explain the difference.

Seminar sheet · for students

# Hashing, passwords and authentication

Cyber security · University · Lecture 3

Name _____ Class _____ Date _____

**1** **A password and one-time code were typed into a fake page. Which factor would have stopped it?**

◯ A longer password ◯ An SMS code ◯ A hardware key bound to the real site

**2** **Estimate an attacker's work.**

A cracker tries $10^9$ fast hashes per second. If each Argon2id guess costs $10^6$ times more, it manages _____ guesses per second. A $10^7$-word dictionary then takes _____ seconds per account.

**Word bank:** 1000 · 1 · 100 · 10000

**3** **Which are sound password-storage practices? Choose all that apply.**

☐ Argon2id with a unique salt per user ☐ SHA-256 with one salt for all users

☐ Checking new passwords against breached lists

☐ Encrypting passwords so admins can read them

**4** **One-time codes sent by SMS fully protect against phishing.**

◯ True ◯ False

**5** **Match each attack to the defence that most directly stops it.**

| | | |
|---|---|---|
| Precomputed rainbow tables | • • • • | Origin-bound hardware keys |
| GPU dictionary attack on a leak | • • • • | A second factor |
| Credential stuffing from other breaches | • • • • | A slow, memory-hard hash |
| Real-time phishing proxy | • • • • | Unique salts |

**How could your students experience the difference between fast and slow hashes safely, without real users' data?**